

Jakarta Server Pages

```
<c:if test="${book.price <= user.preferences.spendingLimit}">  
    The book ${book.title} fits your budget!  
</c:if>
```

```
<c:forEach var="product" items="${products}">  
    <c:out value="Price of ${product.name} is ${product.price}"/>  
</c:forEach>
```

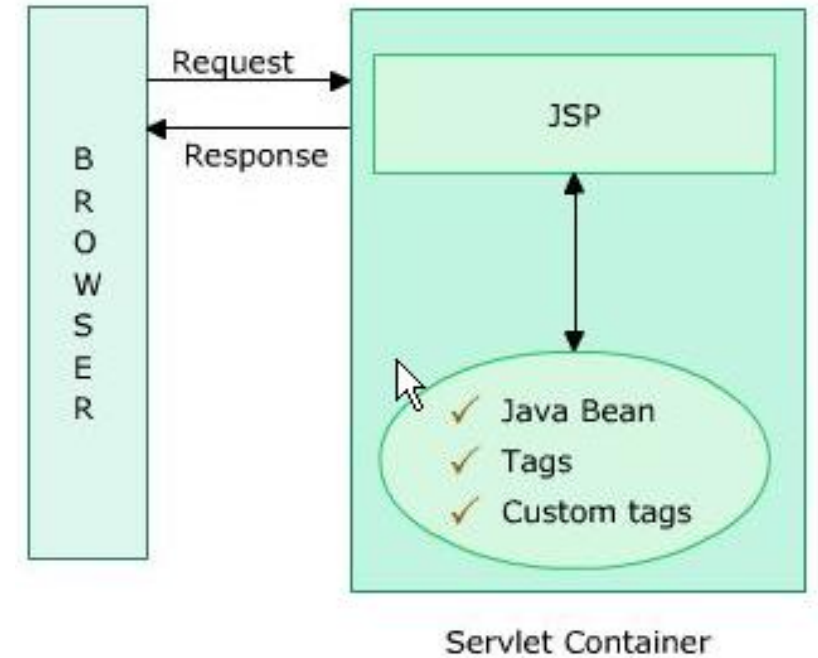
Overview

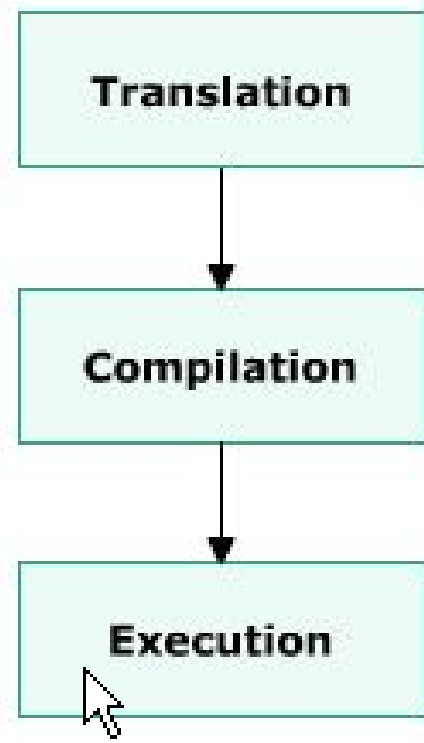
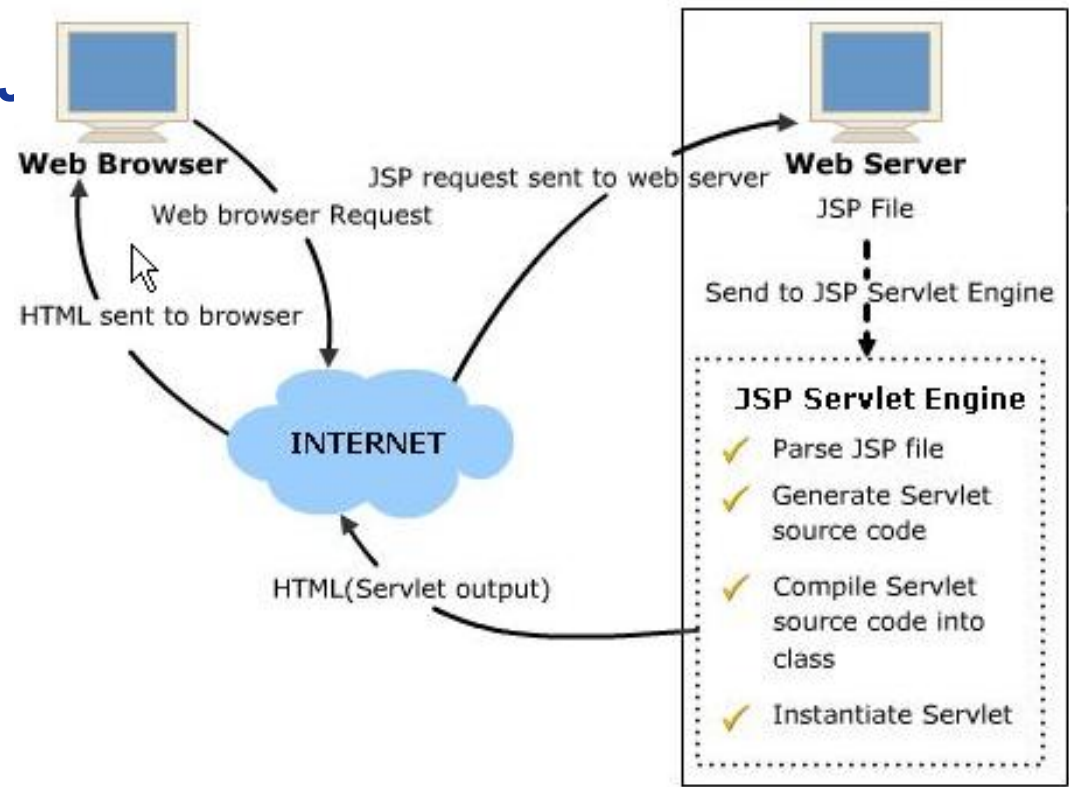
- Jakarta Server Pages (JSP) = a server-side technology for building dynamic web pages.
- Part of Jakarta EE.
- JSPs are compiled into servlets by the container (Tomcat)
- Mix HTML + Java (scriptlets, expressions, declarations).
- Provides implicit objects (request, response, session, etc.).
- Supports directives (page, include, taglib)
- Can use JSTL & Expression Language (EL) for cleaner syntax

Overview

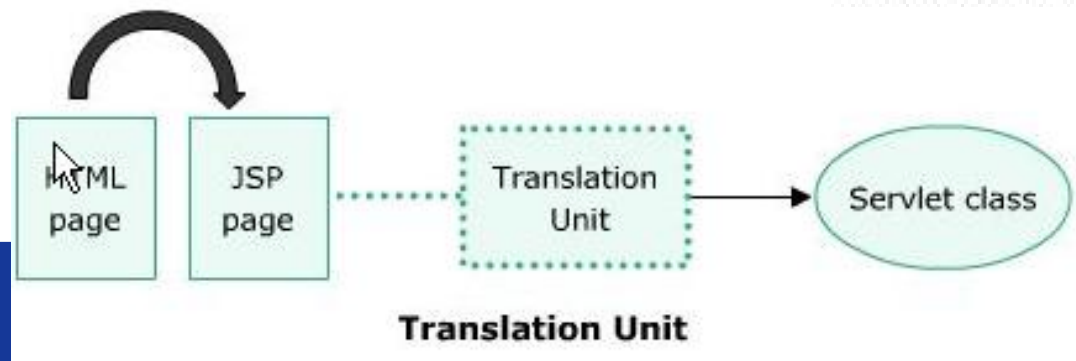
- Life Cycle of JSP:
 1. **Translation** → JSP converted into a servlet.
 2. **Compilation** → Servlet compiled into bytecode.
 3. **Loading & Instantiation** → Servlet loaded by container.
 4. **Request handling** → method called for each request
 5. **Destroy** → Servlet destroyed when app stops.

- Introduction
Jakarta Server Pages
- Jakarta Server Page (JSP) is a server side script language
- Saved with .jsp extension
- A simple, yet powerful Java technology for creating and maintaining dynamic-content webs pages
- JSP page are converted by the web container into a Servlet instance
- It focus on the presentation logic of the web application





JSP Life Cycle



Implicit Objects

Implicit Objects

Object	Type
request	<code>jakarta.servlet.http.HttpServletRequest</code>
response	<code>jakarta.servlet.http.HttpServletResponse</code>
out	<code>jakarta.servlet.jsp.JspWriter</code>
session	<code>jakarta.servlet.http.HttpSession</code>
application	<code>jakarta.servlet.ServletContext</code>
config	<code>jakarta.servlet.ServletConfig</code>
pageContext	<code>jakarta.servlet.jsp.PageContext</code>
page object	<code>jakarta.servlet.jsp.HttpJspPage</code>
exception	<code>java.lang.Throwable</code>

request Object

Method	Return Type	Description
<code>getParameter(String name)</code>	<code>String</code>	Returns the value of a request parameter, or null.
<code>getParameterValues(String name)</code>	<code>String[]</code>	Returns an array of values (e.g., from multi-select checkboxes).
<code>setAttribute(String name, Object o)</code>	<code>void</code>	Stores an attribute in the request scope.
<code>getAttribute(String name)</code> <code>= <i>\${emp.id}</i></code>	<code>Object</code>	Retrieves a previously stored attribute.



Method	Return Type	Description
<code>getHeader(String name)</code>	String	Returns the value of a specified HTTP header.
<code>getCookies()</code>	Cookie[]	Returns an array containing all cookie objects found in the request.
<code>getSession()</code>	HttpSession	Returns the current session, or creates one if it does not exist.
<code>getRequestDispatcher(String path)</code>	RequestDispatcher	Returns an object used to forward the request to another resource.



request Object

- Example:

```
String name = request.getParameter("username");  
String[] hobbies = request.getParameterValues("chkhobbies");  
request.getRequestDispatcher("information.jsp").forward(request, response);
```





response Object

Method	Return Type	Description
setContentType(String type)	void	Sets the MIME type of the response (e.g., text/html).
sendRedirect(String location)	void	Redirects the browser to a new URL.
addCookie(Cookie cookie)	void	Appends a cookie to the response header.
setHeader(String name, String value)	void	Sets a specific response header value.
getWriter()	PrintWriter	Returns a PrintWriter to write character text to the client.



response Object

Example:

```
response.sendRedirect("home.jsp");
```





out Object

- Represents JspWriter for sending output.
- Methods:
 - `print()`,
 - `println()`,
 - `flush()`.
- Example:

```
out.println("<h2>Hello</h2>");
```



session Object

- Represents user session across requests.
- Methods:
 - `setAttribute()`,
 - `getAttribute()`,
 - `invalidate()`.
- Example:

```
session.setAttribute("user", "Alice");
```



application & config

- application (ServletContext): shared across app.
- Example:
 - `application.setAttribute("appName", "Portal");`
 - `config (ServletConfig) : initialization params.`
- Example:
`config.getInitParameter("dbDriver");`



pageContext, page & exception

- pageContext: gives access to all implicit objects.
- page: refers to current JSP (like this).
- exception: only available in error pages.



pageContext

- pageContext: gives access to all implicit objects.
- Can store attributes in different scopes (page, request, session, application)

<%

```
pageContext.setAttribute("x", 100);  
out.println(pageContext.getAttribute("x"));
```

%>



page

- page: refers to current JSP (like this).
- Rarely used directly, but available.

```
<%
```

```
    out.println("This page object: " + page.toString());
```

```
%>
```



exception

- Available **only in error pages** (`isErrorPage="true"`)
- Represents the **Throwable** object that caused the error.

```
<%@ page isErrorPage="true" %>
```

```
<%
```

```
    out.println("Error: " + exception.getMessage());
```

```
%>
```

Directives

Directives

- Directives control the processing of an entire JSP page. It gives directions to the server regarding processing of a page.
- `<%@ directive name [attribute name="value" attribute name="value" ] %>`
- There are three types of Directives in JSP:
 - **Page Directive**
 - **Include Directive**
 - **TagLib Directive**

Directives - Page

- `<%@ page attribute1="value1" attribute2="value2" ...%>`
- **Attributes :**
 - `import; session;`
 - `isErrorPage; errorPage;`
 - `ContentType; isThreadSafe; extends; info;`
 - `language; autoflush; buffer`

Directives – Page

- **Page – import:** attribute of the **page directive** allows a JSP to use Java classes/packages

```
<%@ page import="java.util.List, java.util.ArrayList" %>  
<%  
    List<String> students = new ArrayList<>();  
    students.add("Alice");  
    students.add("Bob");  
    out.println("Students: " + students);  
%>
```


Directives – Page

- **Page – session**: session management
- `<%@ page session="true"%>`: (default) → JSP creates/uses an **HttpSession** object, **session** is available

```
session.setAttribute("user", "Alice");  
out.println(session.getAttribute("user"));
```

- `<%@ page session="false"%>`: No **HttpSession** created, **session** not available

Directives – Page

- **Page – isErrorPage.** Indicates if this page is an error handler
 - `<%@ page isErrorPage="true"%>`
 - `<%@ page isErrorPage= "false"%>`
- **Page – errorPage:** Indicates the page that will handle errors that may occur in this page
 - `<%@ page errorPage="ExceptionHandling.jsp"%>`
- **Page – contentType:** used to set the content type of a JSP page
 - `<%@ page contentType="text/html"%>`

Directives – Include

- **Include: directive** is used to **statically include** another file into a JSP at **translation (compile) time**
- The content of the included file is **copied into the JSP** before it is compiled into a servlet

`<%@include file = "value"%>`

index.jsp

<code><%@include file="header.jsp"%></code>		
<code><%@include file="myJSP.jsp"%></code>		

Directives - taglib

- `taglib` **directive** is used to declare a **custom tag library** in a JSP page.
- It allows JSP to use **custom tags** instead of Java code inside the page Typically used with **JSTL (Jakarta Standard Tag Library)** or custom tag libraries
- `<%@ taglib uri="uri_of_tag_library" prefix="prefixName"%>`

Directives - taglib

```
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
<html>
  <body>
    <c:set var="name" value="Jakarta JSP" />
    <c:out value="${name}" />
  </body>
</html>
```

Jakarta JSP

Actions - jsp:include

- used to include any other resource (may be html, jsp etc) at a runtime.
- Syntax: **<jsp:include page="another file" />**

```
<html>
  <head>
    <title> Main Page </title>
  </head>
  <body>
    <H4> This is main page and to explain include directive </H4>
    <br/>
    <jsp:include page="displayIP.jsp" />
  </body>
</html>
```

Actions - jsp:forward

- used to forward request any other resource (may be html, jsp etc).
- Syntax: **<jsp:forward page="another file" />**

```
<html>
  <head>
    <title> Main Page </title>
  </head>
  <body>
    <H4> This is main page and to explain include directive </H4>
    <br/>
    <jsp:forward page="displayIP.jsp" >
      <jsp:param name="param1" value="value1"/>
      <jsp:param name="param2" value="value2"/>
    </jsp:forward>
  </body>
</html>
```

Actions - jsp:useBean

- Creates a scripting variable associated with a Java object.
- Syntax:

```
<jsp:useBean  
  id= "instanceName"  
  scope= "page | request | session | application"  
  class= "packageName.className"  
</jsp:useBean>
```


Jakarta Server Pages Standard Tag Library 3.0 (JSTL)

<https://jakarta.ee/specifications/tags/3.0/jakarta-tags-spec-3.0#expression-language-overview>

JSTL

- **JSTL**: a collection of **custom tag libraries** for solving common problems such as **iterating over a map or collection**, **conditional testing**, **XML processing**, and even **database access** and **data manipulation**
- To use a JSTL library in a JSP page, use the taglib directive with the following format:

```
<%@ taglib prefix="prefix" uri="uri" %>
```

JSTL

```
<!-- JSTL Jakarta (dùng cho taglib fmt, c:forEach, ...) -->
```

```
<dependency>
```

```
  <groupId>jakarta.servlet.jsp.jstl</groupId>
```

```
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
```

```
  <version>3.0.2</version>
```

```
</dependency>
```

```
<dependency>
```

```
  <groupId>org.glassfish.web</groupId>
```

```
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
```

```
  <version>3.0.1</version>
```

```
</dependency>
```

JSTL

Functional Area	URI	Prefix
core	jakarta.tags.core	c
XML processing	jakarta.tags.xml	x
capable formatting	jakarta.tags.fmt	fmt
relational db access (SQL)	jakarta.tags.sql	sql
Functions	jakarta.tags.functions	fn

JSTL Libraries

1. `<%@ taglib prefix="c" uri="jakarta.tags.core" %>`
2. `<%@ taglib prefix="fn" uri="jakarta.tags.functions"%>`
3. `<%@ taglib prefix="sql" uri="jakarta.tags.sql"%>`
4. `<%@ taglib prefix="fmt" uri="jakarta.tags.fmt"%>`
5. `<%@ taglib prefix="x" uri="jakarta.tags.xml"%>`

Expression Language (EL)

JSP Expression Language (EL)

- Provide another **convenient way** to **minimize** the use of **scripting tag** in JSP
- To **access** the data , **process** the data, **store** it in some scope in JSP
- General syntax: **`${expression}`**

Expression Language Implicit Objects (1)

Name	Description
pageScope-	use to get the value of attribute stored in page scope
requestScope	Use to get the value of attribute stored in request scope.
sessionScope	use to get the value of attribute stored in session scope
applicationScope	use to get the value of attribute stored in application scope.
cookie	use to get the cookie value
initParam	use to get the context init params, we can't use it for servlet init params

Expression Language Implicit Objects (2)

Name	Description
pageContext	provides access to many objects request, session etc.
param	use to get request parameters (valid only for single value)
paramValues	use to get request parameter to an array of values
header	use to get the request header value (valid for single value)
headerValues	use to get the request header value to an array of values.

Expression Language Operators

Operator Type	Description
Arithmetic	+ , - , / or div , * , % or mod
Logical	&& , , !
Relational	== (eq), != (ne), < (lt), > (gt), <= (le) , >= (ge)
Dot (.) Operator	Use to access property of any bean: <code>\${bean.property_name}</code>
[] operator	Use to get the data from array and list along with beans: <code>\${myList[1]}</code> or <code>\${myList["1"]}</code>

Disable Expression Language

```
<%@ page isELIgnored ="true | false" %>
```



Exception Handling

Ways to handle Exceptions – Try Catch

<%

```
try
```

```
    {    }
```

```
catch (Exception    e)
```

```
    {    }
```

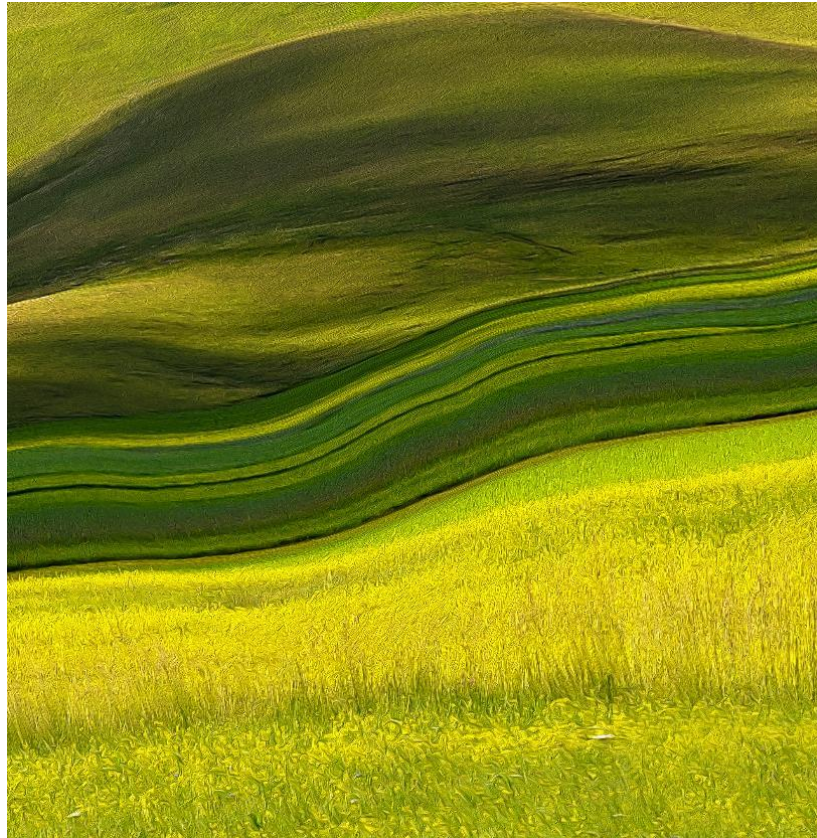
%>

Ways to handle Exceptions – isErrorPage, errorPage

- **isErrorPage:** use to inform container that declaring JSP page is an error page
`<%@ page isErrorPage="true/false" %>`
- **errorPage:** use to set the error page for the JSP
`<%@ page errorPage="jsp page name" %>`

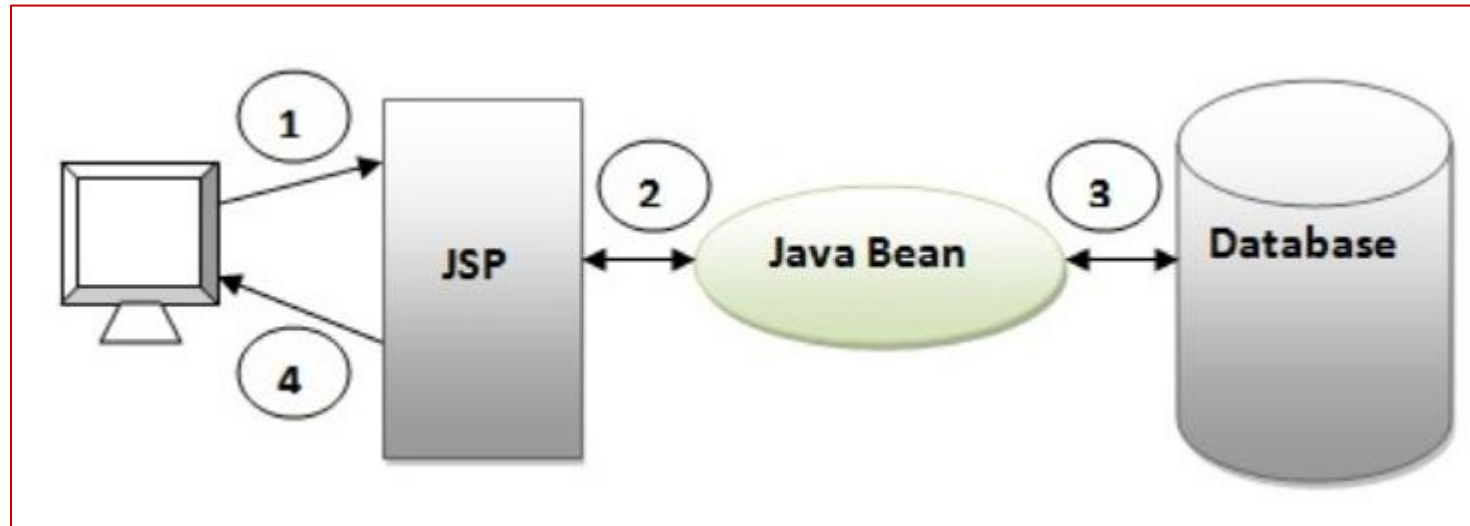


JSP – Application design



Model 1 Architecture

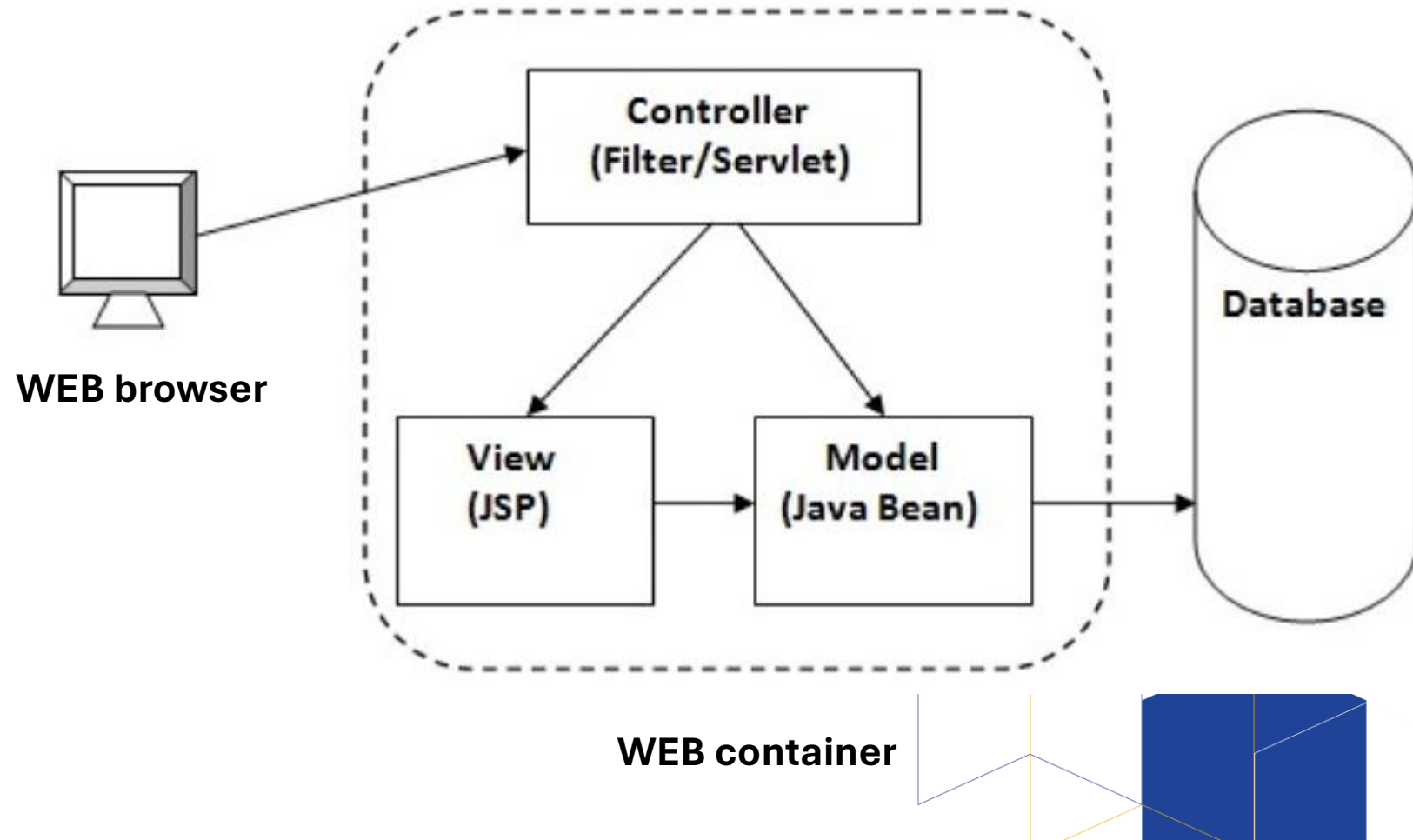
1. Browser sends request for the JSP page
2. JSP accesses Java Bean and invokes business logic
3. Java Bean connects to the database and get/save data
4. Response is sent to the browser which is generated by JSP



Model 1 Architecture

- Easy and Quick to develop web application
- **Navigation control is decentralized:** If JSP page name is changed that is referred by other pages → change it in all the pages that leads to the maintenance problem
- It is better for small applications but not for large applications

MODEL 2-MVC (Model View Controller)

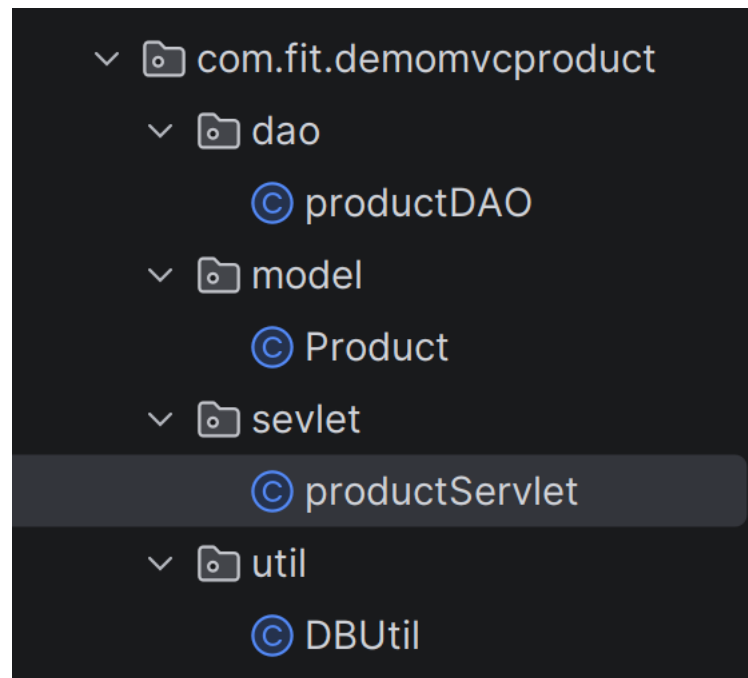


MODEL 2-MVC (Model View Controller)

- Navigation control is centralized. Now only controller contains the logic to determine the next page.
 - Easy to maintain
 - Easy to extend
 - Easy to test
 - Better separation of concerns
-
- We need to write the controller code self.
 - If we change the controller code, we need to recompile the class and redeploy the application

MODEL 2-MVC Example

1. A Product class that is the template for the model objects. An instance of this class contains product information.
2. A ProductForm class, which encapsulates the fields of the HTML form for inputting a product. The properties of a ProductForm are used to populate a Product.
3. A ControllerServlet class, which is the controller of this Model 2 application.
4. An action class named SaveProductAction.
5. Two JSP pages (ProductForm.jsp and ProductDetails.jsp) as the views.
6. A CSS file that defines the styles of the views. This is a static resource.



Bước 1: Thêm các dependencies

```
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>6.0.0</version>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>jakarta.servlet.jsp</groupId>
  <artifactId>jakarta.servlet.jsp-api</artifactId>
  <version>3.1.0</version>
  <scope>provided</scope>
</dependency>
<!-- JSTL -->
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>3.0.1</version>
</dependency>
<dependency>
  <groupId>org.mariadb.jdbc</groupId>
  <artifactId>mariadb-java-client</artifactId>
  <version>3.5.5</version> <!-- bản mới nhất -->
</dependency>
```

- Bước 2: Cấu hình Resource cho connection : Cấu hình `\webapp\META-INF\context.xml`

```
<context>
  <resource

    name="jdbc/productdb"
    auth="Container"
    type="javax.sql.DataSource"
    maxTotal="20"
    maxIdle="10"
    maxWaitMillis="10000"
    driverClassName="org.mariadb.jdbc.Driver"
    url="jdbc:mariadb://localhost:3306/productdb"
    username="root"
    password="root" >

  </resource>
</context>
```

- Bước 3: Model Product.java(id,name, price) sử dụng Lombok

```
private int id; 3 usages  
private String name; 3 usages  
private double price; 3 usages
```

- Bước 4: Kết nối database qua class DBUtil.java

```
public class DBUtil { 3 usages
    private final    DataSource dataSources; 2 usages

    public DBUtil(DataSource dataSource) { 1 usage
        this.dataSources = dataSource;
    }
    public Connection getConnection() throws SQLException { 1 usage
        Connection conn = dataSources.getConnection();
        return conn;
    }
}
```


- Bước 5: ProductDAO.java: Thực hiện các câu query (CRUD) xuống database

```
public class productDAO { 3 usages
    private DBUtil dbutil; 2 usages

    public productDAO(DataSource dataSource) { 1 usage
        dbutil = new DBUtil(dataSource);
    }
```

```
public List<Product> getAllProduct() { 1 usage
    List<Product> lstProduct = new ArrayList<>();
    String sql = "select * from products";
    try {
        Connection con = dbutil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();

        while (rs.next()) {
            Product pro = new Product();
            pro.setId(rs.getInt(columnLabel: "id"));
            pro.setName(rs.getString(columnLabel: "name"));
            pro.setPrice(rs.getDouble(columnLabel: "price"));

            lstProduct.add(pro);
        }
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
    return lstProduct;
}
```

- Bước 6: Tạo Contrller: ProductServlet.java

```
@WebServlet("/products")
public class productServlet extends HttpServlet {

    @Resource(name="jdbc/productdb")

    private DataSource dataSource;

    private productDAO productdao;
```



```
@Override
public void init(ServletConfig servletConfig) throws ServletException {
    try {
        productdao = new productDAO(dataSource);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

```
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException {
    String action = req.getParameter(name: "action");
    if (action==null) action="list";
    switch (action) {
        case "list":
            List<Product> coproduct =productdao.getAllProduct();
            req.setAttribute(name: "products", coproduct);
            req.getRequestDispatcher(path: "listproduct.jsp").forward(req,resp);
            break;
    }
}
```

- Bước 7: Tạo Views: products.jsp \webapp\views\products.jsp

```
<c:forEach items="${products}" var="pro">
    <tr>
        <td>${pro.id}</td>
        <td>${pro.name}</td>
        <td>${pro.price}</td>
        <td>${pro.description}</td>
        <td><a href="${pageContext.request.contextPath}/products?action=detail&id=${pro.id}">Detail</a>
            <a href="${pageContext.request.contextPath}/products?action=edit&id=${pro.id}">Edit</a>
            <a href="${pageContext.request.contextPath}/products?action=delete&id=${pro.id}">Delete</a>
        </td>
    </tr>
</c:forEach>
```



Industrial
University of
Ho Chi Minh City

QUESTIONS